

3.2 입/출력 (Input/Output)

프로그램에 입/출력은 무엇인가요? 사용자로부터 필요한 정보를 받아드리거나 반대로 보여주는 방식으로 사용자 인터랙티브(Interactive) 한 의사소통하기 위한 수단일 것이다.

엔트리 블록코딩에서 우리는 이 두 가지를 다음의 두 블록(을 사용해서 입출력을 코딩해 본 적이 있다.



이제 이를 텍스트코딩으로 구현해보자. 입/출력 기능은 모두 우리가 이전 장에서 배운 Entry 라이브러리 안에 있는 두 종류의 함수(print, input)를 호출함으로써 구현할 수 있다. 함수 이름 자체가 무슨 용도인지 알 수 있을만큼 직관적이다. 해당 함수들은 엔트리-파이썬의 경우 실제 파이썬과는 조금 다르게(실제 파이썬에는 라이브러리 호출 없이 곧바로 사용 가능하며, 이러한 함수들을 특별히 언어자체에 기본으로 내장된 함수라 하여 내장 함수라 부름) Entry라는 라이브러리 안에 존재한다. 이 경우 이 함수들을 어떻게 호출해 사용할 수 있는지를 이전 과에서 이미 배웠고, 라이브러리 이름.함수명 이라는 문법형식으로 호출해야 하는 것을 익히 알고 있다. 따라서, **Entry.print()** 와 **Entry.input()** 으로 호출하면 된다. 다만, 함수가 올바르게 동작하기 위해 호출자로부터 넘겨받아야 할 필수값(파라미터)이 무엇인지만 정확히 알면 되겠다.

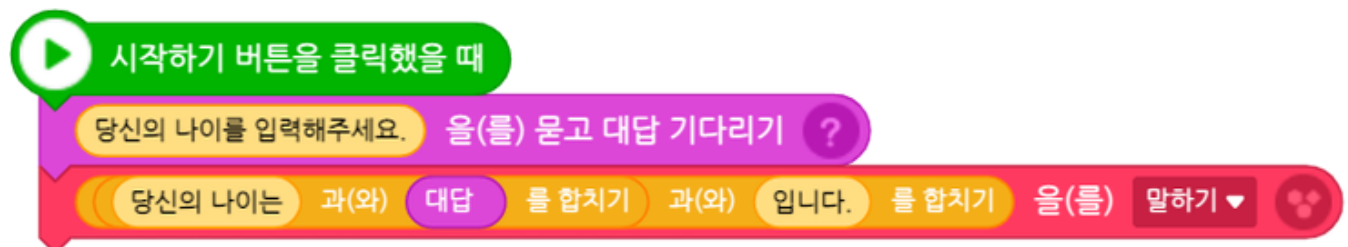
사실 이것도 직관적으로 예상할 수 있듯이 print 함수(출력함수)의 경우, 사용자에게 보여주고자 하는 문구를 넘기고, input(입력함수)의 경우, 사용자로부터 어떤 값을 받아드리고 싶은지 설명하는 문구를 넘겨주면 된다. 이 문구는 사실상 어떠한 문장(텍스트코딩 세계에서는 이러한 데이터의 형식을 스트링(string)이라고 함)이기 때문에 문구를 의미하는 문법표현으로 ""(큰 따옴표) 안에 원하는 내용을 적으면 된다.(작은 따옴표도 되긴 하나 엔트리-파이썬에서는 자동으로 큰 따옴표로 변경된다.) 조금더 재미있는 부분은 출력하고 싶은 문구를 몇개의 문구를 나눈 후 이를 재조립해 출력하는 방법도 가능하다는 것이다. 여러 문구를 조립할 때 문구들 사이에 사용하는 연산자 기호는 + (문구들을 덧붙인다는 직관적 의미를 가짐)이다.

이제 지금까지 이론적으로 배운 부분은 모두 합쳐 사용자의 나이를 입력받고, 출력하는 아주 간단한 프로그램을 텍스트코딩으로 코딩해보자.

실행결과



블록코딩



엔트리-파이썬

```
1 | # Entrybot 오브젝트의 파이썬 코드
2 |
3 | import Entry
4 |
5 | def when_start():
6 |     Entry.input("당신의 나이를 입력해주세요.")
7 |     Entry.print("당신의 나이는" + Entry.answer() + "입니다.")
```

코드를 이해할 수 있겠는가? 조금 낯선 부분이 등장했는데, 바로 `Entry.answer()` 라는 부분이다. 이 함수는 왜 필요할까? 사실 이 함수의 필요는 엔트리-파이썬에만 필요한 함수(실제 파이썬에서는 사용자가 입력한 값을 자신이 직접 정의한 변수에 담아 사용하기 때문에 저런 용도의 함수가 필요하진 않음)로, 함수의 용도는 사용자가 입력한 값을 무엇인지 알아내기 위한 용도로서 `Entry.print` 함수 안에서 출력문장 조립의 한 부분으로서 직접 사용함으로써 사용자의 입력값을 가져다가 출력하는데 활용하였다.

내부적인 동작을 좀더 설명하면 사용자가 어떤 값을 입력하면 그 값을 코드의 다른 부분에서 재활용하기 위해서는 어떻게든 그 값을 어느 공간에 임시로 저장(보통 이를 변수라는 저장공간에 저장하게 됨)을 해두어야 하고, 그 변수를 통해 입력값을 코드의 다른 부분에서 활용하게 된다. 그러나, 우리의 코드는 어디에서 우리가 정의한 변수가 없음에도 원하는 정상동작을 구현했는데, 사실은 엔트리 내부적으로 사용자가 입력한 값을 노출하지 않는 숨겨진 임시변수에 저장해 두고 있고, 우리에게는 단지 그 값을 읽어 올 수 있는 `answer()`라는 함수만을 제공하고 있다.

앞서 언급했듯이 실제 파이썬 코딩에서는 이렇게 활용되지 않는 부분이기도 하나, 엔트리 블록코딩에서 사용자의 편의성을 최대한 끌어올리다보니 불가피하게 발생하는 부분이라고 이해하면 좋겠다. 그리고, 그 블록코딩을 그대로 계승해야 하는 엔트리-파이썬 입장에서조차 불가피한 부분이기도 하다. 엔트리-파이썬은 블록코딩에서 텍스트코딩으로 넘어가기 위한 중간 연결다리로서의 역할이지 파이썬 그 자체를 정확하게 배우기 위한 용도는 아님을 이 예제에서도 알 수 있었다.